Part 1 — How I work with AI

1. My workflow

For this project, I worked mostly with the Pi coding agent in Herdr, using GPT 6.1 Sol with medium thinking. I kept for the most time two agent panes open and prompted both agents myself, while following the code and Git changes in a separate window especially at the beginning of the project. I often fork multiple agents from the same session, preserving the starting context while giving each a separate task and its own evolving context. I keep useful sessions open and close their panes when no longer needed. The agents know about each other and can inspect the other sessions through Herdr, but I decide what to ask and which changes to keep. I used a single Git branch for most of this project and decided which changes to push each time without using different branches. I then switched at the end to different branch and worktrees to test different changes in parallel for example the return part that is later on merged. I also maintained a project skill as a shared guide for the agents: general project rules, where to explore, how to approach the work and where to find the indigo.ai documentation and assignment updated when something changes. I saved the assignment and documentation as local files that the agents could inspect whenever needed, rather than relying only on conversation history.

My approach in building changes depending on what I am building. For ordinary backend code, I work through small changes and tests: does the endpoint return the right data, handle a SOAP error, or recover from an expired session? When the interface will be used by an LLM, that is only half the work. I also need to see whether the model chooses the right tool as intended and in an efficient way, supplies sensible arguments, keeps the customer's constraints and explains the result correctly. In standard backend one doesn't need for example to take into account for the most part the length of the output and the specific format of the responses for example while when the interface is called by a llm this is fondamental. Both the signature and the response format is really important in order to have informations exposed in the correct way and easily retrievable and explainable. For writing prompt i usually use the models themselves and iterate with them exploring both data and during iterations.

Browser Harness, used from Pi, helped me read authenticated documentation, explore the workspace and check the integration in the actual browser. I built a generated question-and-answer testbook and iterated locally with a Pi agent using the same goal prompt and tools derived from the API schema. I treated those generated answers as starting expectations to check, not as proof of correctness. I then tested selected conversations on the deployed chatbot through the widget's socket-backed interface and inspected the matching traces. That let me compare what the assistant said with the tool calls and data behind its answer. Once I had validated the end-to-end flow, I also used application logs to inspect tool calls and results during faster iterations, without repeating the whole conversation every time. Those focused checks helped me iterate, but were not a replacement for end-to-end validation. For the return part i had to step in. In the beginning i let the agent move without actually calling the last soap call to send a return, letting it operate in a sandboxed environment to not discard the 3 orders that can be returned. In the actual final test i stepped in finding bugs that it did not find in the cancel and parallel chats for example.

2. When it went wrong

I have seen even strong models drift from specifications or claim success based on incomplete logs. Their confidence can make errors harder to notice, so I do not treat their reports as proof. The models are continually getting better and are making less and less errors but they are still there. An example is for example in the cited above tests on the concurrent chats and the refunds in the sandbox environment that were not really useful in getting the feature to work.

I can think of different examples happened during the last years when the models were not like the ones we have now but with the latest versions of the models i will mention 2 examples:

As a hobby, I train small models to solve Sudoku or mazes or such toy problems, trying ideas and combinations of papers with coding agents. I have seen results that looked too good to be true, while the agent remained confident after being asked to check again. Information leakage can invalidate those scores: training and test data may overlap or leak for example, or the model may receive information unavailable at prediction time. I learned to inspect the data flow myself with the help of the agents and direct the agent toward specific leakage risks rather than simply ask it to recheck generally.

At work, another trap that i can think of is for example an HTTP 200 that only confirms an asynchronous flow was accepted. Without access to downstream logs, the agent may report success without checking the background processing. Where possible, I add end-to-end tests that wait for and verify the final outcome and tell the agent to inspect all the parts that are included in a flow and the logs, not just the initial response. Otherwise, I mark completion as unverified.

Now I keep the overall goal clear, especially when a primary agent delegates work. I use shared specifications written by a plan with the agent, tests and a blockers file, request concise reports separating evidence from assumptions, and review small changes according to their impact and i constantly challange the agent with everything i can think of. I try to prompt agents myself to understand the work and retain ownership as much as possible and i prompt it everytime to iterate to make the code smaller and simple and making the agent write in technical concise language or generate images of the flow etc to understand everything better and faster. For model-driven features, I combine automated end-to-end tests with manual inspection of conversations and tool calls in selected cases, especially failures or suspicious results. Reviewing every call is not practical, and passing backend tests alone does not prove the assistant followed the customer's request.
Other things that i check are for example if the agents loose the context for example after context compactions or in long contexts chats. I always tell them for example to recheck every now and then the skills and the files and put in the context general informations and so on to have the best performances and latest version of the code(may change if for example multiple agents work in the same code and branch) in the context.

3. When the client's system fights back

My first step would be to explain the problem to the client: what we observed in test, how it differs from the documentation and what it could affect at go-live as fast as possible. I would ask whether they can fix it or whether production behaves like test or already follows the documented contract or a completely different behaviour. Since I cannot access production, I would give their IT team a few targeted calls to run if they don't know the prod behaviours and ask for anonymized results if it is possible for them to run and doesn't violate any policy or data in order to understand the situation also in prod (i cant and wont call prod).

If there is a problem in prod and they cannot change their production system in time or if they don't answer in time in the meantime, I would adapt our integration quickly, provided the change is feasible and safe. I would use a configurable adapter with explicit modes for the confirmed API behaviors and the test behaviour, so we can switch both modes in our test and production environments and on the fly even at the go live . The different modes can be switched fast and on the fly. That would let us switch the compatibility handling without changing the agent's prompt or the client's API. I would test each mode and avoid automatic guesses about which behavior to use mocking both the behaviors of the client since i have the test behaviour and the contract behaviour.

With this adapter in all the cases we can switch to the desired behaviour:
if the client doesn't answer in time
if prod behaves like test
if prod already follows the contract

Before release, I would ask their IT team to verify the selected production mode. I would tell the client what is confirmed and what remains uncertain, and prepare monitoring and rollback.
If all the necessary checks cannot be completed for example if prod doesn't follow neither of the behaviours, we would agree on a reduced release or postpone the affected feature.